Manual de Code Review & Estilo

Engenharia e Arquitetura Mobile • Android (Kotlin) & iOS (Swift)

Versão 2026.1
Guia de Padronização

1. PRINCÍPIOS DE CLEAN CODE & SOLID EM MOBILE

A manutenibilidade de aplicações de alta escala exige estrita observância aos princípios SOLID e Clean Code, adaptados para os ciclos de vida e arquiteturas reativas do Android e iOS.

Single Responsibility (SRP)
ViewModels e Presenters não devem manipular acesso a dados, parsing ou chamadas de sistema (Bluetooth/GPS). Delegue para UseCases/Repositories.

Open/Closed (OCP)
Estenda comportamentos via abstrações e injeção de dependências. Evite estruturas `switch/when` exaustivas sobre enums mutáveis da camada de domínio.

Liskov Substitution (LSP)
Subtipos de Views ou Mappers não devem alterar o contrato da classe base. Prefira composição sobre herança hierárquica profunda.

Dependency Inversion (DIP)
Módulos de alto nível (UI) não dependem de detalhes (Retrofit/URLSession). Ambos devem depender de Interfaces/Protocols desacoplados.

2. CONVENÇÕES DE NOMENCLATURA E FORMATAÇÃO

KOTLIN STANDARD

```
// Classes & Interfaces: PascalCase
class UserProfileRepositoryImpl :
UserProfileRepository

// Funções & Variáveis: camelCase
val isUserAuthenticated: Boolean
fun fetchAccountDetails(id: String)

// Compose UI: PascalCase Noun
@Composable
fun PrimaryButton(onClick: () -> Unit)
```

SWIFT STANDARD

```
// Protocols & Structs: PascalCase
protocol UserProfileRepositoryDelegate:
AnyObject

// API Design Guidelines: Argument Labels
func fetchAccountDetails(for userId: String)

// SwiftUI: PascalCase Noun
struct PrimaryButton: View {
  var action: () -> Void
}
```

3. MATRIZ DE SEVERIDADE PARA REVIEW

Nível	Critério / Impacto	Exemplo de Ação
Bloqueante (P0)	Memory Leak em closures, crash em runtime, vazamento de API Key, violação de LGPD/GDPR.	Recusar PR imediatamente. Correção obrigatória antes de qualquer merge.
Crítico (P1)	Falta de tratamento de erro na camada de rede, violação clara de arquitetura (UI acessando DB).	Exigir refatoração e inclusão de testes unitários cobrindo o cenário.
Maior (P2)	Duplicação de código significativo, acoplamento desnecessário, ausência de testes em UseCases.	Aprovação mediante compromisso de ajuste no mesmo ciclo.
Menor / Nit (P3)	Inobservância de formatação (ktlint/SwiftLint), nome de variável pouco descritivo.	Sugestão não-bloqueante; aplicar correção via linters automáticos no CI.

4. PADRÕES ANTI-LEAK E GESTÃO DE CONCORRÊNCIA

KOTLIN COROUTINES ANTI-PATTERN

```
// ❌ RUIM: GlobalScope vazado
GlobalScope.launch { fetch() }

// ✅ BOM: Structured Concurrency
class UserViewModel : ViewModel() {
  fun loadData() = viewModelScope.launch {
    val data = repository.getData()
  }
}
```

SWIFT CONCURRENCY ANTI-PATTERN

```
// ❌ RUIM: Retain cycle em Task
Task { self.updateUI() }

// ✅ BOM: Capture List com weak self
Task { [weak self] in
  guard let self = self else { return }
  await self.updateUI()
}
```

5. CHECKLIST PARA O REVIEWER

- ✓ **Isolamento de Camadas:** O ViewModel/Presenter não importa pacotes de Framework UI nativo (ex: Android Views ou UIKit)?
- ✓ **Tratamento de Estado:** Os estados de Loading, Success e Error são representados por Sealed Interfaces (Kotlin) ou Enums com Associated Values (Swift)?
- ✓ **Imutabilidade:** Propriedades expostas para a UI usam `StateFlow/Read-only Properties` ao invés de coleções mutáveis diretas?
- ✓ **Testabilidade:** As dependências externas foram injetadas por protocolo/interface permitindo fácil criação de Mocks nos testes?